

AegisQ-HPC: Communication-Aware Distributed Quantum Simulation

with Post-Quantum Secure Job Provenance

Pinar Aksoy

Technical report, September 19, 2026

Abstract

Simulating an n -qubit quantum circuit costs 2^n amplitudes, so beyond roughly thirty qubits the state vector must be partitioned across the memory of several machines. Partitioning turns some quantum gates into network operations, and which gates depends entirely on an arbitrary choice: which logical qubits are assigned to the bits of the basis index that select a rank. We present AegisQ-HPC, a distributed state-vector simulator that makes this choice deliberately, and pairs it with a gate-fusion pass that reduces how often the remaining communicating gates fire. Every byte the runtime hands to MPI is instrumented, an analytical cost model predicts that traffic from the circuit structure, and a search over qubit placements minimises it. On a single 8-core host with shared-memory MPI, communication-aware placement removes 87.2% of measured MPI traffic for a 20-qubit quantum Fourier transform on 8 ranks and 72.3% for a Grover circuit, while leaving a GHZ chain unchanged; the cost model predicted the byte count exactly in all 99 of 99 distributed configurations measured. Wall time on this host is another matter: the circuits the optimiser cannot improve still vary by 30.8% between placements, so we report the traffic reduction as the result and treat wall-time differences below that floor as unresolved. We additionally protect job submission and result provenance with the NIST-standardised post-quantum primitives ML-KEM-768 and ML-DSA-65, and measure that overhead: 1.18% of the runtime of the heaviest job in our suite, of which only $182\mu\text{s}$ is lattice arithmetic. All results are from one laptop; we are explicit throughout about what that does and does not support.

1 Introduction

A pure quantum state of n qubits is a unit vector

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle, \quad \sum_i |\alpha_i|^2 = 1, \quad (1)$$

where we write $|\cdot\rangle$ for a computational basis state. Storing the amplitudes explicitly costs $O(2^n)$: in double precision, 30 qubits is 16 GiB and 40 qubits is 16 TiB. Single-machine simulation therefore becomes memory-bound long before it becomes compute-bound, and the standard response is to distribute the state vector across the memory of many nodes [1, 4].

Distribution introduces a cost that does not exist on one machine. With $P = 2^p$ ranks, each rank holds 2^{n-p} amplitudes and the high p bit positions of a basis index select the rank that owns it. A gate acting on a qubit in one of those positions may have to move amplitudes between ranks. The assignment of logical qubits to bit positions is, however, free: nothing forces qubit q to occupy

position q . The default assignment — the identity permutation — is an accident of indexing, and for some circuits it is close to the worst possible choice.

This report describes a system built around three questions.

RQ1 Can communication-aware qubit placement reduce MPI communication volume and wall-clock time for distributed state-vector simulation?

RQ2 What overhead does authenticating and encrypting HPC quantum jobs and results with standardised post-quantum cryptography introduce?

RQ3 How do small Grover and Shor experiments illustrate the cryptographic motivation for post-quantum cryptography?

The simulator is implemented from scratch in C++20 with MPI and OpenMP. Qiskit [11] is used only as an external correctness oracle in the test suite; it is never imported by the runtime.

2 Background

2.1 State-vector simulation

A single-qubit gate with matrix U acting on qubit q pairs each amplitude α_i with $\alpha_{i \oplus 2^q}$ and applies U to the pair. Enumerating the 2^{n-1} pair representatives touches every amplitude exactly once, which makes the kernel a strided, embarrassingly parallel sweep — the reason state-vector simulation threads well and the reason it is bandwidth-bound.

Two structural properties of a gate determine its behaviour under partitioning. A *diagonal* gate multiplies each amplitude by a scalar that depends only on that amplitude’s index, so it never moves data. A *control* qubit is only read, never written. Both properties turn out to matter more than the gate’s name.

2.2 Distributed-memory simulation

Established distributed simulators partition the state vector across processes and exchange shards for gates acting on the high-order qubits [1, 4, 9]. The usual treatment of placement is to apply gates to whichever qubits they name, occasionally reordering qubits so that the expensive ones become local. We make that reordering the object of study: the runtime carries an explicit logical-to-physical permutation, a cost model predicts its consequences in bytes, and a search chooses it.

2.3 Post-quantum cryptography

NIST standardised module-lattice key encapsulation as ML-KEM (FIPS 203 [8]) and module-lattice signatures as ML-DSA (FIPS 204 [7]) in 2024. We use the reference implementations in liboqs [10] through its Python binding, with HKDF-SHA256 [5] for key derivation and AES-256-GCM [2] for payload encryption. No cryptographic primitive is implemented in this project.

3 System architecture

AegisQ-HPC is four cooperating layers.

Gate	Placement	Bytes (all ranks)	Messages
z s t rz cz	any	0	0
x y h rx ry	local	0	0
x y h rx ry	global	PSw	P
cx	local target	0	0
cx	local control, global target	$PSw/2$	P
cx	global control, global target	$PSw/2$	$P/2$
swap	both local	0	0
swap	one global	$PSw/2$	P
swap	both global	$PSw/2$	$P/2$

Table 1: Communication per gate and placement.

1. A **front end** providing a circuit IR over twelve gates (**x y z h s t rx ry rz cx cz swap**) and a documented OpenQASM subset. Anything outside the subset is rejected with a line number rather than ignored.
2. A **native core** in C++20: local kernels shared by single-process and distributed execution, an MPI runtime, and a communication profiler.
3. A **compiler** comprising an analytical communication cost model, a static placement search, a gate-fusion pass and a windowed placement planner.
4. A **security layer** providing ML-KEM/ML-DSA job envelopes and signed, Merkle-committed result provenance.

A deliberately simple NumPy reference simulator is the correctness oracle for everything else; the C++ and MPI engines are validated against it gate by gate and on randomised circuits.

4 Distributed simulation runtime

4.1 Partitioning

With $P = 2^p$ ranks and n qubits, a physical basis index splits as

$$\text{physical} = (\text{rank} \ll L) \mid \text{local}, \quad L = n - p, \quad (2)$$

so the low L bit positions are *local* and the high p are *global*. A layout object carries a permutation $\pi : [0, n) \rightarrow [0, n)$ mapping logical qubits to bit positions, defaulting to the identity. Every kernel reads placement only through predicates (`is_local`, `global_position`, `partner_rank`), so changing the permutation requires no change to any kernel.

The initial state $|0 \cdots 0\rangle$ is physical index 0 under every permutation and lives on rank 0, so initialisation needs no communication.

4.2 Which gates communicate

Let $S = 2^L$ be the shard size in amplitudes and w the bytes per amplitude. Table 1 summarises the traffic per gate placement; it is the cost model and it is also exactly what the runtime does.

Three entries deserve comment.

Diagonal gates are free even on a global qubit. The rank identifier fixes that qubit’s value for every amplitude the rank holds, so the gate collapses to a single scalar multiply of the whole shard.

A global control is cheap; a global target is not. A control is read, so a rank that does not satisfy it skips the gate entirely and a rank that does proceeds locally. A target is written, and writing a global bit moves amplitudes.

Half-shard exchanges are real, not an approximation. In a `cx` with a local control, only amplitudes whose control bit is set participate. Those form contiguous runs of length $2^{\pi(c)}$, so they are packed with a strided block copy, exchanged, and scattered back — half the bytes of a whole shard exchange.

All pairwise traffic uses a symmetric `MPI_Sendrecv`, which cannot deadlock, into a separate buffer, so no amplitude is overwritten before it is consumed. Buffers are reused across gates and transfers are chunked at 2^{28} elements because MPI element counts are `int`-typed.

4.3 Measurement

Shot sampling draws `shots` uniform values in $[0, 1)$ from a seeded Mersenne Twister, sorts them once, and walks the local amplitudes accumulating probability mass; each rank claims only the draws that fall inside its own interval, whose offset comes from an `MPI_Exscan`. Because the draw sequence depends only on the seed and the shot count, a circuit sampled on 8 ranks produces *identical* counts to the same circuit sampled in one process — a property asserted in the test suite and relied on by the benchmark methodology.

4.4 Instrumentation

Every transfer the runtime issues is timed and counted inside the function that issues it: sends, receives, pairwise exchanges, bytes in each direction, collectives, and a per-opcode breakdown. What is counted is the payload handed to MPI; what is not counted is MPI’s own protocol overhead and the internal traffic of collectives. Reported volumes are therefore a lower bound on wire traffic and an exact account of what the algorithm requested — which is the quantity placement controls.

5 Communication-aware qubit placement

Because byte volume depends only on *which* qubits are global and not on their ordering within each group, the search space is the $\binom{n}{p}$ subsets. For 24 qubits on 8 ranks that is 2024 candidates, so an exhaustive search is affordable and is optimal with respect to the model; beyond a configurable budget the search falls back to a greedy start with pairwise local improvement and reports that it is not guaranteed optimal.

The objective is lexicographic: bytes first, message count as a tiebreak. Message count distinguishes placements that move identical volume in different numbers of transfers, which bytes alone cannot.

The model is not trusted on its own. For every circuit in the evaluation, the predicted byte count is compared against the profiler’s measurement on the live runtime, including under permuted placements and in single precision.

6 Post-quantum job protocol

A job travels as one file so it can move over whatever transport a cluster already has. The client encapsulates to the cluster’s ML-KEM public key, derives an AES key with HKDF bound to the

protocol version and the job identifier, encrypts the manifest and the circuit *together* with AES-256-GCM using the public header as associated data, and signs the header and ciphertext with ML-DSA.

The double binding is deliberate: editing a public field breaks decryption as well as the signature, so a job can never run under parameters other than those that were signed. Everything signed or hashed is serialised canonically (sorted keys, no insignificant whitespace), and the parser re-serialises what it parsed and compares bytes, rejecting input that was not already canonical.

On receipt the cluster checks, in order: structure; the signing key against a local allow-list; the ML-DSA signature; the addressee; replay state; then decapsulation, decryption and the circuit hash. Nothing inside the envelope is trusted before the signature verifies.

Results are committed with a Merkle tree following RFC 6962 [6]: leaves and internal nodes are domain-separated, and odd leaf counts split at the largest power of two rather than duplicating the last leaf, avoiding the ambiguity in which two distinct trees share a root. The manifest — artefact digests, environment, placement, timings — is signed with the cluster’s ML-DSA key.

We state the resulting claim precisely. A valid signature means the record was produced by a holder of the cluster’s secret key and has not been modified. It does *not* mean the computation was performed correctly: a compromised node signs a wrong answer with a valid key. Verifying provenance is not verifying computation.

7 Evaluation

7.1 Setup

All measurements were taken on one Apple M2 laptop (8 logical cores, 16 GiB), macOS 15.6, Apple Clang 17, Open MPI 5.0.10, with shared-memory MPI and no inter-node network. Each configuration runs a discarded warm-up repeat followed by three timed repeats; we report the minimum wall time with the median and spread recorded alongside. Thread policy is an explicit variable: under *one thread per rank* the total core count grows with the rank count (classical scaling), while under *fixed total cores* ranks $\times$ threads is held at the core count, which measures the cost of partitioning rather than the benefit of adding processors. The two are never combined in one table.

7.2 Correctness

The native and distributed engines are validated against the NumPy reference simulator per gate, per placement case, and on randomised circuits at 1, 2, 4 and 8 ranks; the reference is in turn cross-validated against Qiskit on 100 seeded random circuits per run. The suite is run in full before every measurement sweep; its size is reported by the test runner rather than quoted here, since a number typed into prose stops being true the first time a test is added.

7.3 RQ1: communication-aware placement

Table 2 gives the measured effect at 8 ranks. The quantum Fourier transform benefits most: in the textbook decomposition every controlled phase targets the higher-indexed qubit, and the identity placement puts exactly those qubits on the global positions. Moving them local removes 87.2% of the traffic. Its wall time falls by -13.6%, but that change does not clear the noise floor and we do not claim it. Grover’s dense multi-controlled blocks behave similarly. The GHZ chain shows no reduction at all — every qubit but one is a `cx` target exactly once, so no placement is better than

Circuit	MPI bytes sent			Wall time		
	Default (MiB)	Optimized (MiB)	Reduction	Default (ms)	Optimized (ms)	Change
qft	936	120	87.2%	298	257	-13.6% [†]
grover	1792	496	72.3%	468	442	-5.5% [†]
random	392	240	38.8%	136	122	-9.8% [†]
ising	432	368	14.8%	178	152	-14.4% [†]
ghz	24	24	0.0%	15	11	-30.8% [†]

Table 2: Measured MPI traffic and wall time under the default and communication-aware placements, 20 qubits on 8 ranks. Byte counts are exact. Wall-time changes marked [†] are unresolved: they do not exceed the noise floor of 30.8%, measured on the circuits whose traffic the optimiser leaves unchanged and whose true effect is therefore zero.

another. We report that as a result: a heuristic that claimed a win on every circuit would be the suspicious one.

Wall time does not track bytes proportionally. The Ising chain sheds 14.8% of its traffic with no measurable time improvement, which is what one expects when communication is not the bottleneck — here, shared-memory MPI on one socket, where a “transfer” is a memory copy.

The cost model predicted the byte count and the message count *exactly* in all 99 distributed configurations measured.

7.4 When is the search necessary?

The cost rules are worth reading for what they depend on. A non-diagonal single-qubit gate costs a full shard if *its* qubit is global; a **cx** costs half a shard if *its target* is global, irrespective of the control, because the byte volume is the same either way. Both depend on one qubit’s membership of the global set. **swap** is the exception: it costs if *either* operand is global, and a disjunction is not a sum.

It follows that for a circuit with no **swap** gates the objective is linear in the indicator of the global set, and the optimum is the p qubits with the smallest individual costs — obtainable by sorting, with no search at all. The mapper detects this case and reports it as such rather than running a search it does not need.

Where **swap** gates appear the problem is combinatorial and the greedy fallback could settle for a local optimum. Across 99 sampled configurations at 4, 8 and 16 ranks, 93 of which required the heuristic, it matched the exhaustive optimum in every case while running 3–40× faster. That is an empirical result on these circuit families rather than a guarantee, and it is reported as one.

7.5 A second lever: gate fusion

Placement decides which gates communicate; it cannot reduce how often a given qubit is acted on. A run of consecutive single-qubit gates on a global qubit costs one whole-shard exchange *per gate*, although the run’s product is a single 2×2 matrix. Multiplying the run out before execution replaces those exchanges with one.

We carry the fused matrix in full rather than as Euler angles. There is no hardware target to compile for, and the explicit form is exact including the global phase, so fused and unfused circuits are compared amplitude by amplitude rather than through a fidelity. Two further properties are

Circuit	Baseline (MiB)	Fusion only	Placement only	Both
qft	936	0.0%	87.2%	87.2%
grover	1792	16.1%	72.3%	86.6%
random	392	8.2%	38.8%	53.1%
ising	432	0.0%	14.8%	14.8%
ghz	24	0.0%	0.0%	0.0%

Table 3: Measured reduction in MPI traffic from each optimisation lever at 8 ranks, 20 qubits.

checked by property-based tests over arbitrary circuits: fusion never increases predicted communication, and a run of diagonal gates fuses into a diagonal matrix that the runtime and the cost model both still recognise as free — a fused gate is classified by inspecting its matrix, not its opcode.

Table 3 gives the measured 2×2 . The levers are not additive. For random circuits fusion alone removes 8.2% of traffic and placement alone 38.8%, while the combination removes 53.1%: fusing changes the cost landscape, so the placement search that follows finds a different and better assignment. For the QFT, whose single-qubit runs are already short, fusion contributes nothing and placement does all the work.

7.6 A third lever: windowed placement

A single assignment is the right answer only for a circuit whose communication structure is uniform. We split a circuit into windows, find the best assignment for each, and switch when the saving on a window exceeds the cost of the switch — half a shard per rank for each qubit that changes group.

A plan is executed as a circuit rewrite rather than a runtime feature: gates are re-expressed on slots, a change of assignment becomes SWAP gates between slots, and the resulting traffic is measured by the same instrumentation as every other gate. The final permutation is undone by relabelling results, which is free classical bookkeeping, exactly as the runtime mapping is for a static assignment.

Two details determine whether the comparison means anything. First, the baseline must be the best *static* assignment, not the default one; measuring against the default would credit windowing with the gain ordinary placement already delivers. Second, the plan must not be charged for restoration that the static baseline gets for free. Both were wrong in our first implementation, and both showed up as windowing apparently losing to static placement on circuits where it plainly should not.

Measured at 8 ranks, windowed placement beats the best single assignment on four of five families: 95.7% of baseline traffic removed for the QFT against 87.2% static, and 87.1% against 72.3% for Grover. On GHZ, which has no phase structure, the planner declines to switch and the two coincide. For random circuits the best result comes from placement and fusion together (53.1%) rather than from windowing (42.9%), so no single lever dominates.

Table 4 gives strong scaling with one thread per rank. Efficiency falls from 92% at 2 ranks to about 30% at 8, on a machine with 4 performance and 4 efficiency cores and no separate interconnect. Under the fixed-total-cores policy the same QFT reaches only $0.87\times$ — that is, on constant hardware, partitioning costs more than it saves, which is the honest framing of what MPI buys on a single node.

Circuit	Qubits	Ranks	Wall (s)	Speedup	Efficiency
ising	22	1	1.815	1.00	100%
ising	22	2	0.961	1.89	94%
ising	22	4	0.681	2.66	67%
ising	22	8	0.750	2.42	30%
qft	22	1	5.211	1.00	100%
qft	22	2	2.765	1.88	94%
qft	22	4	1.907	2.73	68%
qft	22	8	1.988	2.62	33%

Table 4: Strong scaling with one thread per rank.

7.7 Are the local kernels efficient?

Wall time says whether a run was fast, not whether the implementation was good. State-vector simulation is bandwidth-bound, so each kernel is compared against what the same machine achieves on an in-place scale of an array of the same type, compiled with the same flags and threaded the same way. The in-place shape is the comparable one: a gate reads and writes a single array, which is not the traffic shape of a two-array copy.

At 8 threads and 24 qubits the kernels that sweep the whole state — a general single-qubit gate and a diagonal one — reach 98% of that reference. They are memory-bound, and there is little left to win. The kernels that touch only part of the state (**cx**, **cz**, **swap**) plateau between 52% and 59%: they read and write a strided fraction of the array and leave roughly half the available bandwidth unused. That is a concrete optimisation target rather than an unexplained gap.

The same sweep tests an assumption the cost model makes. The model treats the local/global distinction as the only thing that matters and ignores which local position a qubit occupies. Measured across target positions, that holds for the full-sweep kernels and for **cx** (spread under 6%), and fails for **cz**, whose achieved bandwidth varies by 57% with the target position. The model is therefore accurate about network traffic and incomplete about local cost.

7.8 RQ2: cost of the post-quantum layer

Table 5 gives primitive costs. End to end, packing a 20-qubit job bundle takes 3.1 ms and verifying, decrypting and opening it 5.6 ms, with 7083 bytes of envelope overhead independent of circuit size. Against the heaviest job in our suite (582 ms, 1792 MiB of MPI traffic) that is 1.18% of runtime.

The decomposition matters more than the total: only 182 μ s of that is lattice arithmetic. The remainder is canonical JSON serialisation and base64 encoding of the circuit. A reader who concluded “post-quantum cryptography is expensive” from the end-to-end number would be drawing the wrong lesson; the serialisation is what an optimisation would need to target.

7.9 RQ3: why post-quantum cryptography

Table 6 reports simulated Grover searches. Every row is a run, not a formula: the measured probability of finding the marked state agrees with the amplitude-amplification prediction to within a few tenths of a percent, and 256 items are searched in 12 oracle queries against a classical expectation of 128.5.

That quadratic factor [3] is precisely why post-quantum guidance doubles symmetric key sizes rather than abandoning symmetric cryptography. Shor’s exponential advantage [12] is a different matter, and our implementation illustrates it at a size that makes the point without overstating

Algorithm	Operation	Median (μs)	Bytes
ML-DSA-44	keygen	32.9	1312
ML-DSA-44	sign	63.2	2420
ML-DSA-44	verify	30.8	2420
ML-DSA-65	keygen	53.6	1952
ML-DSA-65	sign	94.9	3309
ML-DSA-65	verify	49.4	3309
ML-DSA-87	keygen	91.2	2592
ML-DSA-87	sign	136.6	4627
ML-DSA-87	verify	84.3	4627
ML-KEM-1024	decapsulate	27.6	32
ML-KEM-1024	encapsulate	24.1	1568
ML-KEM-1024	keygen	22.9	1568
ML-KEM-512	decapsulate	14.5	32
ML-KEM-512	encapsulate	12.8	768
ML-KEM-512	keygen	12.0	800
ML-KEM-768	decapsulate	19.9	32
ML-KEM-768	encapsulate	17.4	1088
ML-KEM-768	keygen	16.8	1184

Table 5: Post-quantum primitive cost and artefact sizes.

N	Qubits	Classical queries	Grover queries	Measured success
4	2	2.5	1	100.0%
8	4	4.5	2	95.4%
16	6	8.5	3	96.6%
32	8	16.5	4	99.9%
64	10	32.5	6	99.7%
128	12	64.5	8	99.6%
256	14	128.5	12	100.0%

Table 6: Grover oracle queries against classical search, measured by simulation.

it: the order-finding circuit factors 15 and 21. Controlled modular multiplication is synthesised generically — the map is a permutation of basis states, decomposed into transpositions, each realised by a CNOT ladder and one multi-controlled X — which is exponential in the register width and therefore stops where the simulation does.

8 Limitations

Every measurement here comes from a single laptop with shared-memory MPI. There is no inter-node network, so communication times are best-case; on a real interconnect the placement optimisation should matter *more*, but that is a prediction, not a result. Problem sizes are 20–23 qubits. There is no GPU backend and no GPU was available. We make no performance comparison against other simulators, because a fair comparison requires care about which optimisations each applies.

The simulator is exact and noiseless, supports twelve gates and terminal measurement only, and requires power-of-two rank counts. The cost model counts bytes and messages and does not model topology, congestion, or overlap of computation with communication. Placement is chosen once per circuit; dynamic re-mapping between circuit segments is future work.

The security layer is a research prototype. A malicious cluster administrator, memory disclosure during execution, side channels (including the communication volume this project deliberately exposes), denial of service, key distribution and revocation are all outside the threat model.

9 Future work

Three directions follow directly from what is measured here. *Dynamic re-mapping*: splitting a circuit into windows and re-permuting when the predicted saving exceeds the permutation cost, which the cost model already has the machinery to decide. *Cluster measurements*: the same experiments on multiple nodes, where communication is a network rather than a memory copy and the byte reductions reported here should translate into larger time reductions. *Topology awareness*: extending the cost model beyond byte counts to account for the interconnect a job is actually running on. The measured interaction between placement and fusion also suggests that the two should be searched jointly rather than applied in sequence.

10 Conclusion

Partitioning a quantum state vector forces a choice that is usually made by accident. Making it deliberately — with a cost model in bytes, a search over placements, and instrumentation that measures what actually crossed the network — removed up to 87% of MPI traffic on the circuits we measured, and the model predicted the byte counts exactly in every distributed configuration. Protecting the same jobs with NIST-standardised post-quantum cryptography cost about 1.5% of runtime, dominated by serialisation rather than lattice arithmetic. Both numbers come from one laptop, and the repository says so in every place they appear.

References

- [1] Koen De Raedt, Kristel Michiels, Hans De Raedt, Binh Trieu, Guido Arnold, Marcus Richter, Th. Lippert, H. Watanabe, and N. Ito. Massively parallel quantum computer simulator. *Computer Physics Communications*, 176(2):121–136, 2007.
- [2] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac. Technical Report NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007.
- [3] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 212–219, 1996.
- [4] Tyson Jones, Anna Brown, Ian Bush, and Simon C. Benjamin. QuEST and high performance simulation of quantum computers. *Scientific Reports*, 9:10736, 2019.
- [5] Hugo Krawczyk and Pasi Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). Technical Report RFC 5869, Internet Engineering Task Force, 2010.
- [6] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. Technical Report RFC 6962, Internet Engineering Task Force, 2013.
- [7] National Institute of Standards and Technology. Module-lattice-based digital signature standard. Technical Report FIPS 204, U.S. Department of Commerce, 2024.

- [8] National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Technical Report FIPS 203, U.S. Department of Commerce, 2024.
- [9] NVIDIA Corporation. cuQuantum SDK and CUDA-Q: distributed state-vector simulation. <https://developer.nvidia.com/cuquantum-sdk>, 2024.
- [10] Open Quantum Safe Project. liboqs: C library for prototyping and experimenting with quantum-resistant cryptography. <https://openquantumsafe.org/>, 2024.
- [11] Qiskit contributors. Qiskit: An open-source framework for quantum computing. <https://www.ibm.com/quantum/qiskit>, 2024.
- [12] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.